

DDPO Reward Calibration — Test Methodology & Results

ddpm-jax · validating the physics rewards before DDPO finetuning
Notebook: base_results/reward_calibration.ipynb · Module: src/rewards.py · 6 July 2026

1. Why calibrate at all — the threat model

DDPO **maximizes the reward**. That single fact forces two pre-conditions on the reward, and this study exists to check both before a single training step is taken:

1. **It will exploit any weakness (Goodhart / reward hacking)**. If a reward can be driven up by a change that does **not** improve the flow, DDPO will find that change. So we must know, in advance, which components can be fooled and by what — and ensure at least one component catches each hack.
2. **It can only steer if it ranks quality in the regime the models actually occupy**. Our models sit in a narrow band (MSE 0.141–0.144, hi- k retention 0.35–0.37). A reward that separates GT from garbage but is flat **within** that operating band cannot generate a useful gradient once training starts.

Concretely we run four tests, each answering one question:

test	question it answers	§
Monotonicity	Does each distance actually grow as the flow degrades — i.e. does it measure what we think it measures?	2
Hack-resistance	Which components can be fooled by a spectrum-matched fake, and which catch it?	3
Separation & ranking	Does the reward rank the real models by quality, and does the signal survive per-input baselining?	4
Cross-Re stability	Are the component scales stable enough across Re for one weighting to work?	5

Setup common to all tests. Everything runs on CPU/numpy (the TPU is busy with inference), streaming reconstruction pkls one at a time. Three sample populations recur: (i) **GT triplets** from held-out Re=1000 sequences; (ii) the **five real model reconstructions** — base + 4 conditional adapters — on Re=1000 seqs 32–33; (iii) **OOD base reconstructions** at Re=500 and Re=2000. Reward anchors are built from **training-side** frames disjoint from every evaluation set, so no test scores a sample against its own anchor data.

2. Test 1 — Monotonicity along degradation ladders

Procedure. Take clean GT triplets and corrupt them along four **ladders** of increasing strength, each a controlled caricature of a real failure mode. At every rung, evaluate all reward distances, then measure the rank correlation (Spearman ρ) between degradation strength and distance. A reward that measures what it claims should rise **monotonically** along the ladders it polices.

ladder	real failure it simulates
blur (σ)	The model’s actual over-smoothing — a Gaussian blur of the field.
lowpass (k_{cut})	Idealized over-smoothing: a hard spectral cut removing all energy above k_{cut} .
noise (σ_n)	Spurious high- k energy — the pathology of the sparse NN-fill input .
phase-scramble (k_0)	Randomize the phases of shells $\geq k_0$ while keeping $ \hat{\omega} $ bit-exact . Preserves the spectrum, destroys the dynamics. $k_0 = 0$ = “spectrum-matched noise”, the canonical hack.

Result (Spearman ρ of mean distance vs. degradation strength; $|\rho| \geq 0.9$ = monotone):

ladder	spec	spec_highk	energy	w1	pde	pde_lr
blur	1.00	0.90	0.00	0.00	0.70	0.60
lowpass	1.00	0.96	−1.00	−0.96	0.79	0.79
noise	1.00	1.00	1.00	1.00	1.00	1.00
scramble	0.49	0.34	0.23	−0.83	1.00	1.00

Reading. No single component is monotone on **everything**, and it shouldn't be — each polices a subset of failure modes and is deliberately blind to the rest (a level metric can't see scale redistribution; a spectral metric can't see phases). Judge each on its own beat:

- spec / spec_highk — clean on **blur**, **lowpass**, **noise** (the shape/energy failures they exist to catch). Flat on **scramble** by construction: scrambling preserves the spectrum, so a spectral distance **cannot** move. That blind spot is the whole reason pde exists.
- energy, w1 — monotone only on **noise**. They are scale-blind (energy) and spatially-blind (w1) global guards, not shape sensors; the negative ρ on lowpass just means removing high- k energy **lowers** the total, which they correctly track downward.
- pde / pde_lr — **perfectly** monotone on **scramble** ($\rho = 1.00$), the one ladder the statistical rewards can't see. This is the complementary coverage the combination relies on.

Two honest warts. spec_highk dips below 0.9 on blur only because at the extreme rung ($\sigma = 4$) the high- k energy underflows the float32 relative floor and saturates — spec (full band) covers that corner. And raw pde is only weakly monotone on blur ($\rho = 0.70$) because mild smoothing **removes** the GT's own truncation noise; this is exactly why DDPO uses the log-ratio form pde_lr (§6 of the math spec), which penalizes departing the GT floor in either direction.

3. Test 2 — Hack-resistance (the phase-scramble probe)

This is the adversarial test that matters most. **Phase-scrambling** builds the worst case for a statistics-matching reward: a field whose spectrum equals the reference **exactly**, but whose phases — and therefore all spatial structure and dynamics — are random noise. If a reward scores this near its GT floor, DDPO could drive that reward to its optimum by generating noise.

Procedure. Scramble every shell ($k_0 = 0$) of GT triplets and report each distance as a multiple of that component's GT floor.

	spec	spec_highk	energy	w1	pde	pde_lr
distance / GT floor	1.0×	1.0×	1.0×	0.9×	109×	146×

Reading. Every statistical component sits **at its floor** — spectrum-matched noise is invisible to all of them (w1 even dips slightly **below**, because random phases Gaussianize the vorticity PDF toward the reference). Only pde_lr fires, and it fires by **two orders of magnitude**. **This single result dictates the reward design:** the spectral/statistical rewards cannot be used alone — DDPO would hack them with noise — and pde_lr is the mandatory anti-hack term that makes the combined reward safe. A sample can only win the combination by matching the statistics **and** obeying the equation.

4. Test 3 — Separation and quality-ranking on the real models

Procedure. Compute every component per frame on the five real reconstructions (base + 4 conditional, Re=1000 seqs 32–33) and on GT. Two things are checked: do the distances lift the recons clearly above the GT floor (sensitivity), and do they **rank** the models by the quality metric we care about, hi- k retention (steering).

Separation (mean per-frame distance; GT is the target floor):

model	spec	sp_hik	energy	w1	pde	pde_lr	hi-k ret
GT	0.47	0.58	0.012	0.064	13.5	3.14	1.00
base	1.11	1.39	0.034	0.078	30.1	0.36	0.484
grad_frozen60	1.13	1.42	0.032	0.078	28.5	0.32	0.482
grad_full60	1.18	1.49	0.032	0.078	31.8	0.40	0.466
field_frozen60	1.11	1.39	0.032	0.077	30.5	0.37	0.485
field_full60	1.20	1.51	0.032	0.078	29.0	0.31	0.450

The spectral distances put every recon at 5–6× the GT floor — ample separation from ground truth. But the five models are nearly identical (MSE within 2% by construction), so the **between-model** spread is tiny. That raises the real question for DDPO, which is not “recon vs GT” but “can the reward tell a slightly better sample from a slightly worse one **of the same input?**”

4.1. The key finding: the signal is real but hidden by anchor variance

Procedure. Rank-correlate each distance against hi-*k* retention three ways: **pooled** over all frames; **within-input** (per frame, across the 5 models — treating the models as 5 samples of the same input); and at **model level** (the 5 model means).

component	pooled per-frame ρ	within-input ρ	model-level ρ
spec	0.11	−0.74	−1.00
spec_highk	0.08	−0.70	−1.00
energy	0.02	0.12	0.20
w1	−0.16	0.06	0.00
pde_lr	−0.28	0.07	0.40

(negative = distance falls as hi-*k* retention improves — the reward points the right way.)

Read the spec row across: pooled $\rho = 0.11$ looks **broken** — as if the reward were unrelated to quality. But within-input it is $\rho = -0.74$, and at model level a **perfect** $\rho = -1.00$. The signal was there all along; the pooled number buries it. **Why, and why it forces per-input baselines:** the reward is anchored to a **regime average**, so each input carries a systematic offset — how far that input’s own ground truth sits from the regime mean. That offset is common to all samples of one input and dominates the pooled variance, drowning the quality signal. It **cancels** the moment you compare samples **of the same input**. This is the empirical proof that DDPO’s per-input advantage baseline (subtracting each input’s group-mean reward) is not a variance-tuning nicety here but a **correctness requirement** — without it the spectral reward looks like noise; with it, it ranks quality perfectly.

Note also that energy, w1, and pde_lr do **not** rank hi-*k* retention even within-input — they measure other things (total level, PDF shape, dynamics). They earn their place as guards and anti-hack terms, not as the steering signal; the steering comes from the spectral pair.

5. Test 4 — Cross-Re scale stability

Procedure. At Re = 500 / 1000 / 2000, measure each component’s GT floor and the distance scale of an actual recon population, and report their ratio (how far recons sit above GT). This sets the per-Re normalization s_i written to reward_calibration.json.

recon / GT-floor	spec	spec_highk	energy	w1	pde	pde_lr
Re = 500	11.6×	14.2×	5.0×	1.9×	12.1×	12.3×
Re = 1000	5.7×	5.9×	1.8×	0.8×	1.5×	2.2×

recon / GT-floor	spec	spec_highk	energy	w1	pde	pde_lr
Re = 2000	61.7×	96.9×	42.6×	2.0×	0.8×	0.6×

Reading. The spectral deficit is **enormous** at Re=2000 (60–97× the floor, vs. 6× in-distribution) — i.e. the OOD regime is exactly where the reward has the most to pull, which is encouraging for using it to drive OOD generalization. The scales differ by an order of magnitude across Re, which is **why** the reward divides each component by a per-Re s_i before weighting, so one set of weights transfers. **Wart to fix before Re=2000 DDPO:** the pde floor there varies strongly by sequence, so its recon/floor ratio (0.6–0.8×) is dominated by anchor noise — the residual anchor needs per-sequence (or per-input) estimation at high Re before it can be trusted.

6. Verdict and how it configures DDPO

component	monotone (policed)	scramble /floor	recon /floor	within-in. ρ	role in DDPO
spec	PASS	1.0×	5.7×	−0.74	steering (full cascade)
spec_highk	PASS*	1.0×	5.9×	−0.70	steering (deficit band)
pde_lr	PASS	146×	2.2×	0.07	anti-hack (dynamics)
energy	PASS	1.0×	1.8×	0.12	weak guard (level)
w1	PASS	0.9×	0.8×	0.06	weak guard — candidate to drop

* spec_highk fails blur only at the $\sigma = 4$ float32-saturation rung; spec covers it.

The calibration resolves the reward configuration for the DDPO loop:

- **Use the combination, never a single component.** Spectral pair steers (it uniquely ranks the hi- k deficit), pde_lr guards (it uniquely catches the phase hack). Neither is safe alone.
- **pde in log-ratio mode** (pde_lr, anchored to the GT residual floor) — raw pde mildly rewards blur.
- **Per-input advantage baselines are mandatory** — the regime anchor injects a per-input offset that only cancels within an input (the 0.11 → −0.74 collapse).
- **Starting weights** spec 0.5 · spec_highk 1.0 · energy 0.25 · w1 0.25 · pde 1.0, each divided by the per-Re scale s_i from reward_calibration.json. w1 is the weakest sensor (recons **below** its GT floor) and may be dropped; energy stays as a cheap level guard.
- **Known follow-ups:** the spec_highk float32 saturation at extreme blur, and the noisy Re=2000 pde anchor — fix the latter (per-sequence residual reference) before OOD DDPO at Re=2000.

Companion: docs/ddpo_reward_math.{md,typ,pdf} (reward definitions) · base_results/reward_calibration.ipynb (the runnable study, all numbers here trace to its cells) · base_results/reward_calibration.json (scales/floors/weights consumed by rewards.make_ddpo_reward).